

Software Testing Syllabus

Module 1

Basics of Software Testing: Basic definitions, Software Quality , Requirements Behaviour and Correctness, Correctness versus Reliability, Testing and Debugging, Test cases, Insights from a Venn diagram, Identifying test cases, Test-generation Strategies, Test Metrics, Error and fault taxonomies , Levels of testing, Testing and Verification, Static Testing. **Problem Statements:** Generalized pseudocode, the triangle problem, the NextDate function, the commission problem, the SATM (Simple Automatic Teller Machine) problem, the currency converter, Saturn windshield wiper

T1:Chapter1, T3:Chapter1, T1:Chapter2.

Text Books:
1. Paul C. Jorgensen: Software Testing, A Craftsman's Approach, 3 rd Edition, Auerbach Publications, 2008. (Listed topics only from Chapters 1, 2, 5, 6, 7, 9, 10, 12, 13)
2. Mauro Pezze, Michal Young: Software Testing and Analysis – Process, Principles and Techniques, Wiley India, 2009. (Listed topics only from Chapters 3, 4, 16, 17, 20,21, 22,24)
3. Aditya P Mathur: Foundations of Software Testing, Pearson Education, 2008.(Listed topics only from Section 1.2 , 1.3, 1.4 ,1.5, 1.8,1.12,6. 2.1,6. 2.4)

Module -1

Basics of Software Testing

Q 1.) Define the terms: (i) error (ii) fault (iii) failure (iv) incident (v) test case

Error:

A good synonym of Error is *mistake*. When mistakes are done while coding, these are called as *bugs*.

Errors tend to propagate. An example is a requirements error may be magnified during design and amplified still more during coding.

Fault:

A fault is the result of an error. A fault is the representation of an error, where representation is the mode of expression, such as narrative text, dataflow diagrams, hierarchy charts, source code, and so on.

Defect is a good synonym for fault.

There are two categories of faults:

1. **A fault of commission** occurs when we enter something into a representation that is incorrect.
2. **Faults of omission** occur when we fail to enter correct information. Of these two types, faults of omission are more difficult to detect and resolve.

Failure:

A failure occurs when a fault executes.

Two subtleties arise here:

1. Failures only occur in an executable representation, which is usually taken to be source code or loaded object code.
2. Second definition relates failures only to faults of commission.
 - How to deal with failures that correspond to faults of omission?
 - What about faults that never happens to execute, or do not execute for a long time? Reviews prevent many failures by finding faults and well-done reviews can find faults of omission.

Incident:

An incident is the symptom associated with a failure that alerts the user to the occurrence of a failure.

Test case:

- Test case has an identity and is associated with a program behavior.
- A test case also has a set of inputs and expected outputs.

Test:

- A test is the act of exercising software with test cases.
- A test has two distinct goals: to find failures and to demonstrate correct execution.

Q 2.) Write a note on testing life cycle with diagram

Figure 1.1 portrays a life cycle model for testing. In the development phases, there are three opportunities arise for errors to be made, resulting in faults that propagate through the remainder of the development process.

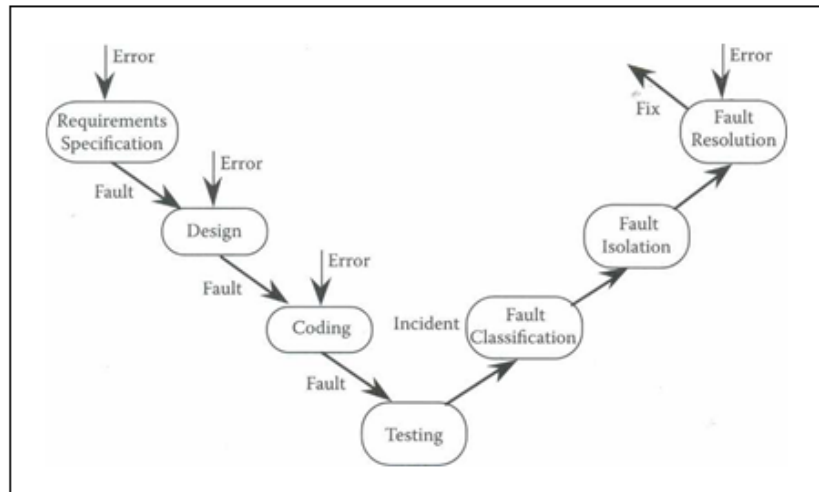


Figure 1.1: A testing life cycle

This life cycle is summarized as follows:

The first three phases are Putting Bugs IN

The testing phase is Finding Bugs

The last three phases are Getting Bugs OUT

The Fault Resolution step is another opportunity for errors (and new faults).

When a fix causes formerly correct software to misbehave, the fix is deficient.

The process of testing can be subdivided into separate steps:

- Test planning
- Test case development
- Running test cases
- Evaluating test results

Q) What is test case? Clarify what information should be in a test case.

Software testing is the process to determine a set of test cases for the item to be tested.

A test case should contain the following information:

1. **Inputs:** There are two types:

Preconditions: Circumstances that hold prior to test case execution

Actual inputs: These are identified by some testing method

2. **Expected outputs:** There are two types:

Postconditions

Actual outputs

3. Test cases should have an **identity** and a **reason for being**.
4. The execution **history** of a test case, including **when** and **by whom** it was run, the pass/fail **result** of each execution, and the **version** (of software) on which it was run.

Test Case ID
Purpose
Preconditions
Inputs
Expected Outputs
Postconditions
Execution History
Date Result Version Run By

Figure 1.2: Typical test case information.

The act of software testing involves establishing the necessary preconditions, providing the test case inputs, observing the outputs, comparing these with the expected outputs, and then ensuring that the expected postconditions exist to determine whether the test passed.

Q 3.) Explain the fundamental approaches used to identify test cases.

Q) Explain functional testing. What are the advantages and disadvantages of functional testing?

Q) Write the frequently suffered problems by functional test cases

Q) Explain structural testing. What are the advantages and disadvantages of structural testing.

Two fundamental approaches are used to identify test cases,

1. Functional Testing
2. Structural testing.

Functional Testing

Functional testing considers any program to be a function that maps values from its input domain to values in its output range. This leads to the term **black box testing**, in which the content (implementation) of a black box is not known, and the function of the black box is understood completely in terms of its inputs and outputs as shown in Figure 1.3.

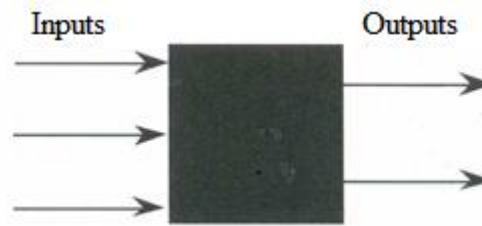


Figure 1.3: An engineer's black box

In the functional approach to test case identification, the only information used is the specification of the software.

Functional test cases have two distinct advantages:

1. They are independent of how the software is implemented, so if the implementation changes, the test cases are still useful
2. Test case development can occur in parallel with the implementation, thereby reducing the overall project development interval.

Functional test cases frequently suffer from two problems (Disadvantages):

1. Significant redundancies may exist among test cases
2. Compounded by the possibility of gaps of untested software

Figure 1.4 shows the results of test cases identified by two functional methods. Method A identifies a larger set of test cases than does Method B. As we noticed, for both methods, the set of test cases is completely contained within the set of specified behavior. Because functional methods are based on the specified behavior, it is hard to imagine these methods identifying behaviors that are not specified.

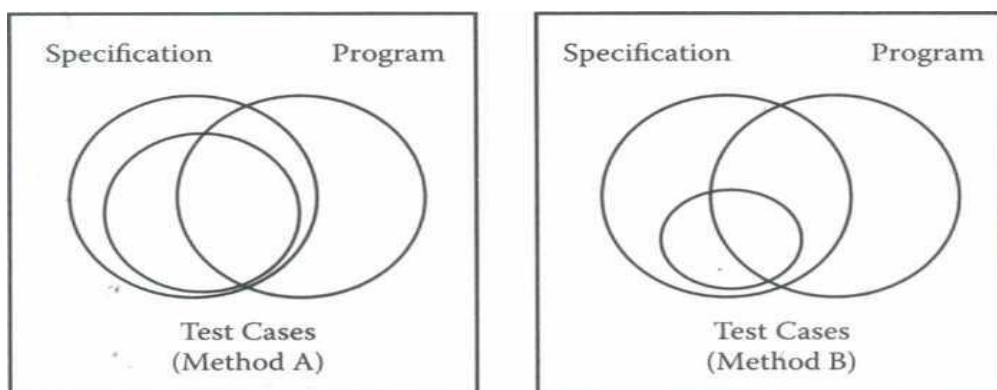


Figure 1.4 Comparing functional test case identification methods

Structural Testing

- Structural testing knows the implementation and this information is used to identify test cases. It is called **white box or clear box** testing.
- The ability to "see inside" the black box allows the tester to identify test cases based on how the function is actually implemented.

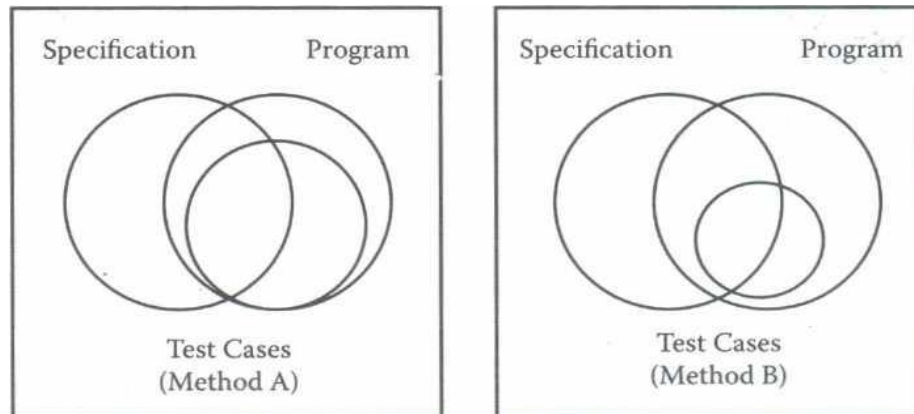


Figure 1.5 Comparing structural test case identification methods.

As in Figure 1.5, for both methods, the set of test cases is completely contained within the set of programmed behavior. Because structural methods are based on the program, it is hard to imagine these methods identifying behaviors that are not programmed.

- A set of structural test cases is relatively small with respect to the full set of programmed behaviors.
- Structural testing uses test coverage metrics, these provide a way to explicitly state the extent to which a software item has been tested, and this in turn makes testing management more meaningful.

Structural test cases have three distinct advantages:

1. Structural testing forces test developer to reason carefully about implementation
2. Structural testing helps to reveals errors in "hidden" code
3. Structural testing spots the Dead Code or other issues with respect to best programming practices.

Structural test cases have three distinct disadvantages:

1. It requires knowledge of the code to perform test.
2. It requires training in the tool used for testing.
3. Sometimes it is expensive.

The Functional versus Structural Test cases

Q 4.) What is Software testing? Differentiate between functional testing and structural testing.

Software Testing is a method to check whether the actual software product matches expected requirements and to ensure that software product is Defect free.

Table 1.1: Differences between Functional Testing and Structural Testing

Functional Testing	Structural Testing
Also known as black box testing	Also known as White box testing
Uses the specification to derive the test data	Uses the program code to derive the test data
Testing is performed by the software testers	Testing is performed by the software developers
Compares program performance with the program specification	Does not test the software against the program specification
Functional testing is the standard test method to verify the functional behavior of a system-under-test.	Does not show that all software components have been developed

Q 5.) Write Error and Fault Taxonomies

The definitions of error and fault axis on the distinction between process and product: process refers to how we do something, and product is the end result of a process.

The point at which testing and software quality assurance (SQA) meet is tries to improve the product by improving the process.

Figure 1.6 contains a fault taxonomy that distinguishes faults by the severity of their consequences

1. Mild	Misspelled word
2. Moderate	Misleading or redundant information
3. Annoying	Truncated names, bill for \$0.00
4. Disturbing	Some transaction(s) not processed
5. Serious	Lose a transaction
6. Very serious	Incorrect transaction execution
7. Extreme	Frequent "very serious" errors
8. Intolerable	Database corruption

9. Catastrophic	System shutdown
10. Infectious	Shutdown that spreads to others

Figure 1.6 Faults classified by severity.

Q 6.) List out different IEEE standard anomalies.

Some of the more useful anomalies are given in Table 1.2 through Table 1.6

Table 1.2: Input/ Output Faults

Type	Instances
Input	Correct input not accepted
	Incorrect input accepted
	Description wrong or missing
	Parameters wrong or missing
Output	Wrong format
	Wrong result
	Correct result at wrong time (Too early/Too late)
	Incomplete or missing result
	Spurious result
	Spelling/grammar
	Cosmetic

Table 1.3: Logic Faults

Missing Case(s)
Duplicate case(s)
Extreme condition neglected
Misinterpretation
Missing condition
Extraneous condition(s)
Test of wrong variable
Incorrect loop iteration
Wrong operator

Table 1.4: Computation Faults

Incorrect algorithm
Missing compilation
Incorrect operand
Incorrect operation
Parenthesis Error
Insufficient precision
Wrong built-in function

Table 1.5: Interface Faults

Incorrect interrupt handling
 I/O timing
 Call to wrong procedure
 Call to nonexistent procedure
 Parameter mismatch
 Incompatible types
 Superfluous inclusion

Table 1.6: Data Faults

Incorrect initialization
 Incorrect storage/access
 Wrong flag/index value
 Incorrect packing/unpacking
 Wrong variable used
 Wrong data reference
 Scaling or units error
 Incorrect data dimension
 Incorrect subscript
 Incorrect type
 Incorrect data scope
 Sensor data out of limits
 Off by one
 Inconsistent data

Q 7.) Explain Levels of Testing

Levels of testing echo the levels of abstraction found in the waterfall model of the software development life cycle. Although this model has its drawbacks, it is useful for testing as a means of identifying distinct levels of testing and for clarifying the objectives that pertain to each level.

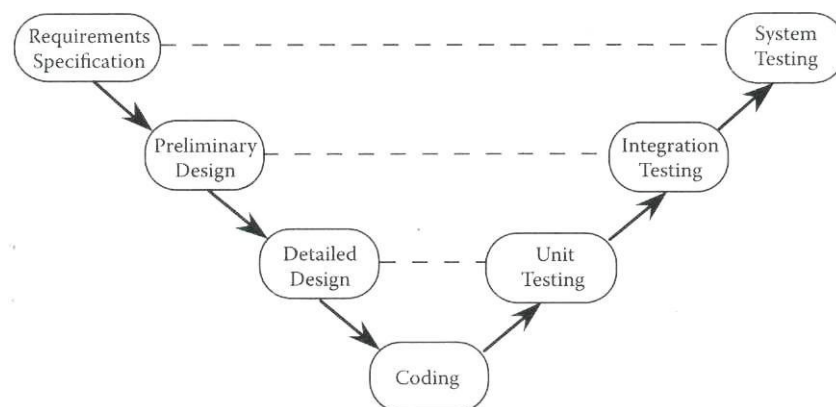


Figure 1.7 Levels of abstraction and testing in the waterfall model.

A diagrammatic variation of the waterfall model is given in Figure 1.7; this variation emphasizes the correspondence between testing and design levels. In terms of functional testing, the three levels of definition (specification, preliminary design, and detailed design) correspond directly to three levels of testing — system, integration, and unit testing.

A practical relationship exists between levels of testing versus functional and structural testing.

Most practitioners agree that structural testing is most appropriate at the unit level, while functional testing is most appropriate at the system level.

The constructs defined for structural testing make the most sense at the unit level, and similar constructs are only now becoming available for the integration and system levels of testing.

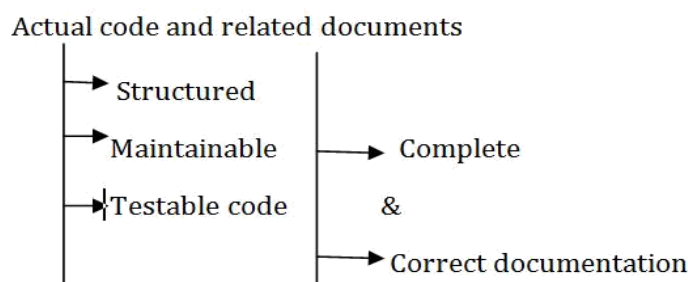
SOFTWARE QUALITY

Q 8.) Define Software quality. Brief out different quality attributes

Software quality is a multidimensional quantity and is measurable.

Quality Attributes can be divided to static and dynamic quality attributes.

Static quality attributes: It refers to the actual code and related documents.



Example: A poorly documented piece of code will be harder to understand and hence difficult to modify.

A poorly structured code might be harder to modify and difficult to test.

Dynamic quality Attributes:

- Reliability
- Correctness
- Completeness
- Consistency
- Usability performance

Reliability:

- It refers to the probability of failure free operation.

Correctness:

- Refers to the correct operation and is always with reference to some artefact.

- For a Tester, correctness is w.r.t to the requirements
- For a user correctness is w.r.t the user manual

Completeness:

- Refers to the availability of all the features listed in the requirements or in the user manual.
- An incomplete software is one that does not fully implement all features required.

Consistency:

- Refers to adherence to a common set of conventions and assumptions.
- Ex: All buttons in the user interface might follow a common-color coding convention.

Usability:

- Refer to ease with which an application can be used. This is an area in itself and there exist techniques for usability testing.
- Usability testing is a testing done by its potential users.
- The development organization invites a selected set of potential users and asks them to test the product.
- Users in turn test for ease of use, functionality as expected, performance, safety and security.
- Users thus serve as an important source of tests that developers or testers within the organization might not have conceived.
- Usability testing is sometimes referred to as user-centric testing.

Performance:

Refers to the time the application takes to perform a requested task. Performance is considered as a non-functional requirement.

Reliability:

- Software reliability is the probability of failure free operation of software over a given time interval & under given conditions or in its intended environments.
- The term environment refers to the software and hardware elements needed to execute the application. These elements include the operating system (OS) hardware requirements and any other applications needed for communication.

Testing and Debugging

Q 9.) Explain testing and debugging with a neat diagram.

- Testing is the process of determining if a program behaves as expected. In the process one may discover errors in the program under test.

- However, when testing reveals an error, the process used to determine the cause of this error and to remove it is known as debugging. As illustrated in figure 1.8, testing and debugging are often used as two related activities in a cyclic manner.

Steps are

1. Preparing a test plan
2. Constructing test data
3. Executing the program
4. Specifying program behavior
5. Assessing the correctness of program behavior
6. Construction of oracle

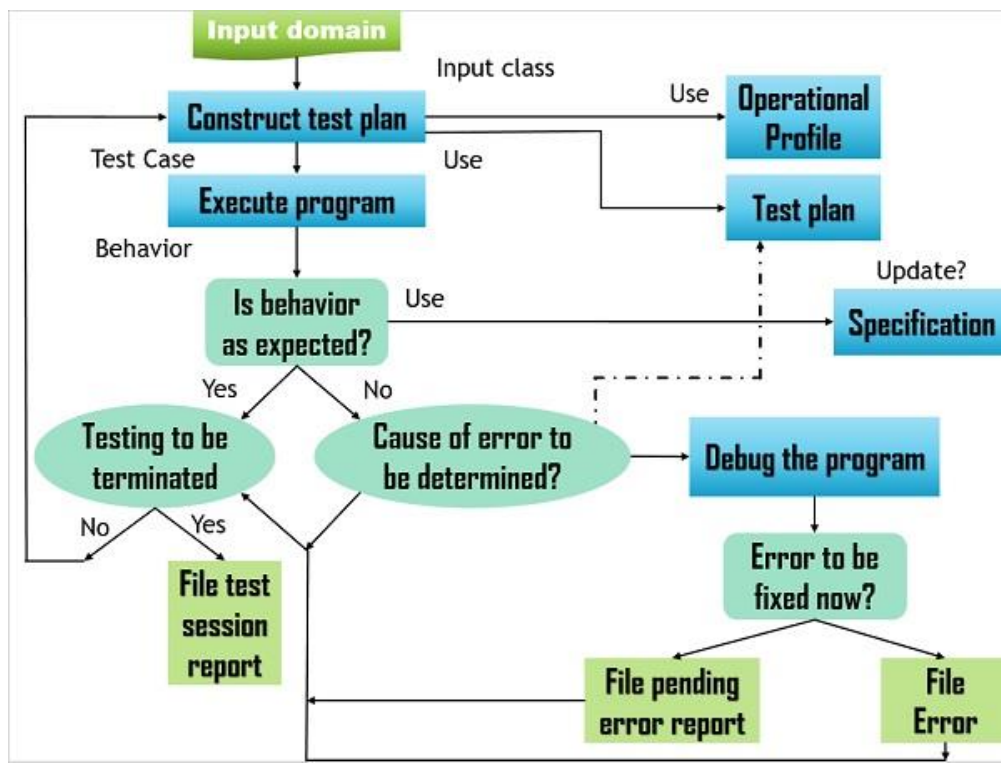


Figure 1.8: Testing and Debugging process

Preparing a test plan:

A test cycle is guided by a test plan. When relatively small programs are being tested, a test plan is usually informal and in the tester's mind or there may be no plan at all.

Example test plan: Method used for testing, method for evaluating the adequacy of test cases, and method to determine if a program has failed or not.

Test plan for sort:

The sort program is to be tested to meet the requirements given in example

1. Execute the program on at least two input sequence one with “A” and the other with “D” as request characters.
2. Execute the program on an empty input sequence
3. Test the program for robustness against erroneous input such as “R” typed in as the request character.
4. All failures of the test program should be recorded in a suitable file using the company failure report form.

Constructing Test Data:

- A test case is a pair consisting of test data to be input to the program and the expected output.
- The test data is a set of values, one for each input variable.
- A test set is a collection of zero or more test cases.

Based on the results obtained, the testers decide whether to continue the construction of additional test cases or to enter the debugging phase.

The following test cases are generated for the sort program using the test plan in the previous figure.

Test case 1:

Test data: < "A" 12 -29 32 >

Expected output: -29 12 32

Test case 2:

Test data: < "D" 12 -29 32 >

Expected output: 32 12 -29

Test case 3:

Test data: < "A" . >

Expected output: No input to be sorted in ascending order

Test case 4:

Test data: < "D" . >

Expected output: No input to be sorted in descending order

Test case 5:

Test data: < "R" 3 17 . >

Expected output: Invalid request character

Valid characters: "A" and "D"

Test case 6:

Test data: < "A" c 17 . >

Expected output: Invalid number

Executing the program:

- Execution of a program under test is the next significant step in the testing.
- Testers might be able to construct a test harness to aid in program execution. The harness initializes any global variables, inputs a test case, and executes the program. The output generated by the program may be saved in a file for subsequent examination by a tester.

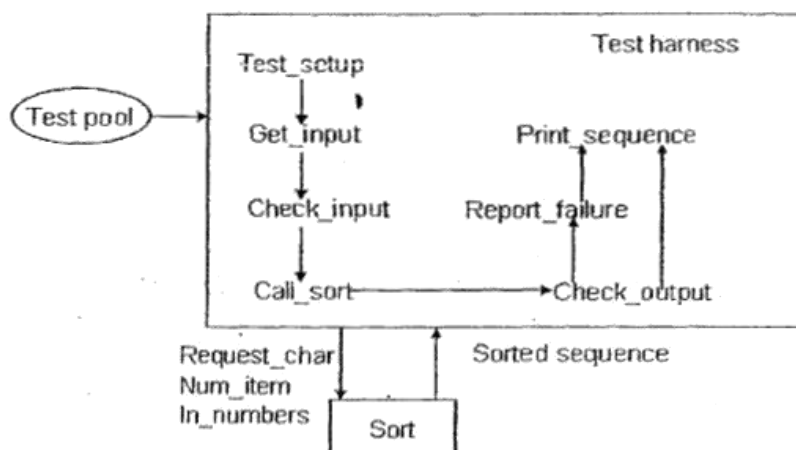


Figure 1.9: A simple test harness to test the sort program

In preparing this test harness assume that:

- Sort is coded as a procedure
 - The get-input procedure reads the request character & the sequence to be sorted into variables request char, num_ items and in number, test setup procedure-invoked first to set up the test includes identifying and opening the file containing tests.
- Check output procedure serve as the oracle that checks if the program under test behaves correctly.
 - Report failure: output from sort is incorrect. May be reported via a message(or) saved in a file.
 - Print_sequence: prints the sequence generated by the sort program. This also can be saved in file for subsequent examination.

- **Specifying program behavior:** The simplest way to specify the program behavior in a natural language as English.

State of the program can be used to define program behavior.

The state of a program is the set of current values of all its variables and an indication of which statement in the program is to be executed next.

State vector: collecting the current values of program variables into a vector known as the state vector.

An indication of where the control of execution is at any instant of time can be given by using an identifier associated with the next program statement.

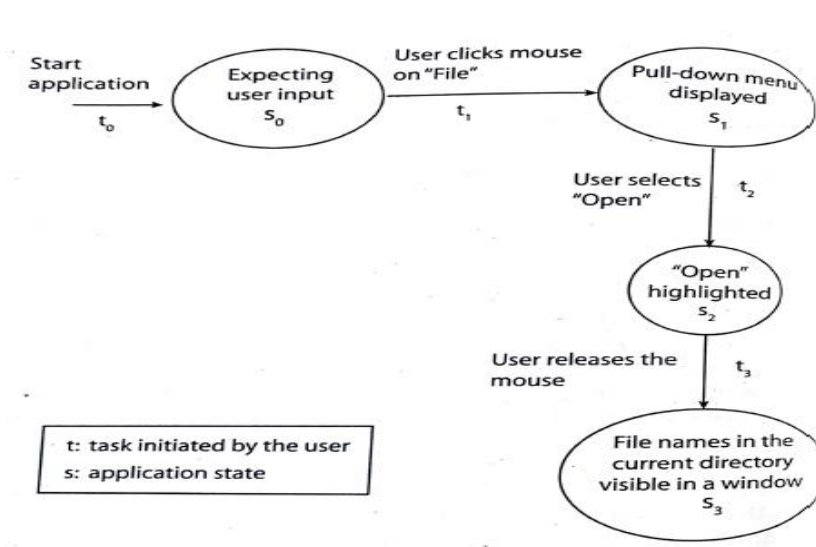


Figure 1.10: A state sequence for myapp showing how the application is expected to behave when the user selects the open option under the File menu item.

State sequence diagram can be used to specify the behavioral requirements. This same specification can then be used during the testing to ensure if the application confirms to the requirements.

Assessing the correctness of program behavior

The tester determines if the observed behavior of the program under test is correct or not.

Behavior: It has two steps:

1. Observes the behavior
2. Analyzes the observed behavior to check if it is correct or not.

The entity that performs the task of checking the correctness of the observed behavior is known as an **oracle**.

But human oracle is the best available oracle.

- Oracle can also be programs designed to check the behavior of other programs.

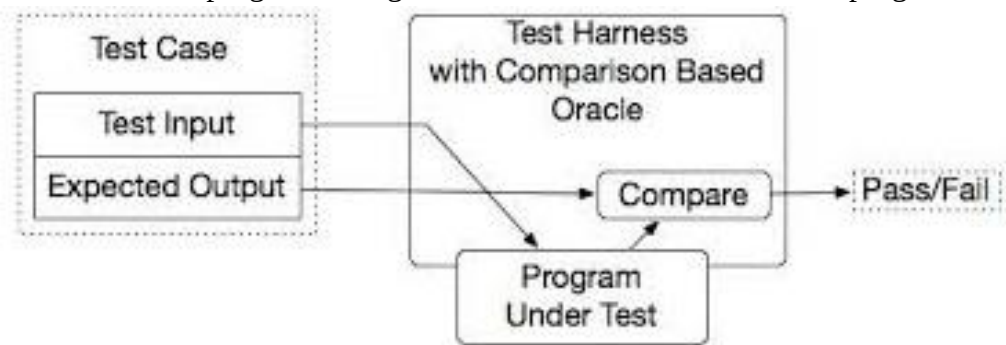


Figure 1.11: Relationship between the program under test and oracle

Construction of oracles:

- Construction of automated oracles, such as the one to check a matrix multiplication program or a sort program, Requires determination of I/O relationship. When tests are generated from models such as finite-state machines (FSMs) or state charts, both inputs and the corresponding outputs are available. This makes it possible to construct an oracle while generating the tests.
- Example: Consider a program named Hvideo that allows one to keep track of home videos. In the data entry mode, it displays a screen in which the user types in information about a DVD. In search mode, the program displays a screen into which a user can type some attribute of the video being searched for and set up a search criterion.
- To test Hvideo we need to create an oracle that checks whether the program function correctly in data entry and search nodes. The input generator generates a data entry request. The input generator now requests the oracle to test if Hvideo performed its task correctly on the input given for data entry.
- The oracle uses the input to check if the information to be entered into the database has been entered correctly or not. The oracle returns a pass or no pass to the input generator.

TEST METRICS

- The term metric refers to a standard of measurement. In software testing, there exist a variety of metrics.

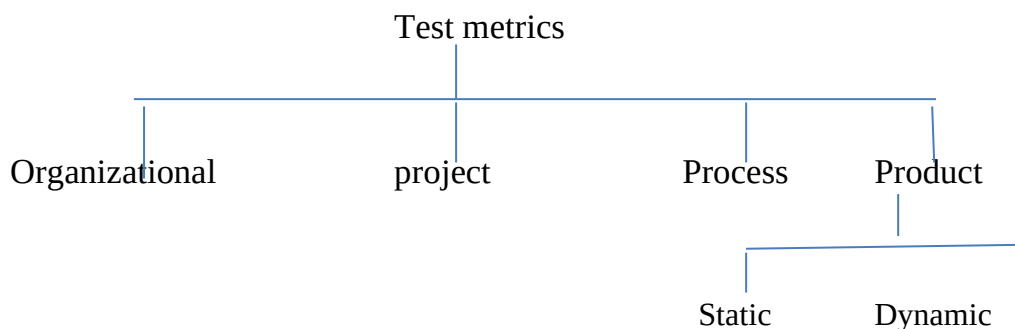


Figure 1.12: Types of test metrics used in software testing and their relationships

There are four general core areas that assist in the design of metrics as schedule, quality, resources and size.

Schedule related metrics:

Measure actual completion times of various activities and compare these with estimated time to completion.

Quality related metrics:

Measure quality of a product or a process

Resource related metrics:

Measure items such as cost in dollars, man power and test executed.

Size-related metrics:

Measure size of various objects such as the source code and number of tests in a test suite

Organizational metrics:

Metrics at the level of an organization are useful in overall project planning and management.

Ex: the number of defects reported after product release, averaged over a set of products developed and marketed by an organization, is a useful metric of product quality at the organizational level.

- Organizational metrics allow senior management to monitor the overall strength of the organization and points to areas of weakness. Thus, these metrics help senior management in setting new goals and plan for resources needed to realize these goals.

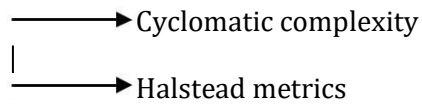
Project metrics:

- Project metrics relate to a specific project, for example the I/O device testing project or a compiler project
- These are useful in the monitoring and control of a specific project
- The ratio of actual to planned system test effort is one project metrics. Test effort could be measured in terms of the tester_man_months

Process metrics:

- The goal of a process metric is to assess the goodness of the process
- Test process consists of several phases like unit test, integration test, system test, one can measure how many defects were found in each phase. It is well known that the later a defect is found, the costlier it is to fix.

Product metrics: Generic- Product metrics related to a specific product such as a compiler for a programming language.



Cyclomatic complexity- based on control flow of the program

$$V(G) = E - N + 2P$$

Program p containing N node, E edges and p connected procedures.

Larger value of V(G) higher program complexity & program more difficult to understand & test than one with a smaller value.

V(G) of the values 5 or less are recommended

Halsted Metrics- Using program size S and effort E, the following estimator has been proposed for the number of errors B found during a software development effort

Table 1.7; Halstead measures of program complexity and effort

<u>Measure</u>	<u>Notation</u>	<u>Definition</u>
Operator count	N_1	Number of operators in a program
Operand count	N_2	Number of operands in a program
Unique operators	η_1	Number of unique operators in a program
Unique operands	η_2	Number of unique operands in a program
Program vocabulary	η	$\eta_1 + \eta_2$
Program size	N	$N_1 + N_2$
Program volume	V	$N \times \log_2 \eta$
Difficulty	D	$2 / \eta_1 \times \eta_2 / N_2$
Effort	E	DXV

Product metrics: OO software metrics are reliability, defect density, defect severity, test coverage, cyclomatic complexity, weighted methods/class, response set, number of children.

TEST GENERATION STRATEGIES

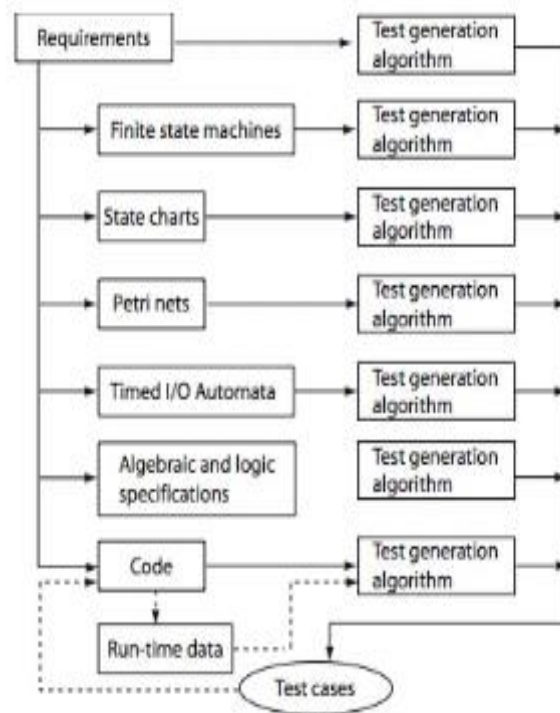


Figure 1.13: Requirements, models and test generation algorithms

Any form of test generation uses a source document. In the most informal of test methods, the source document resides in the mind of the tester who generates tests based on a knowledge of the requirements.

The tests are generated using a mix of formal and informal methods either directly from the requirements document serving as the source. In more advanced test processes, requirements serve as a source for the development of formal models.

Model based: require that a subset of the requirements be modeled using a formal notation (usually graphical).

Models: Finite State Machines, Timed automata, Petri net, etc.

Specification based: require that a subset of the requirements be modeled using a formal mathematical notation.

Examples: B, Z, and Larch.

Code based: generate tests directly from the code.

EXAMPLES

Three examples will be used to illustrate the various unit testing methods.

1. The triangle problem
2. A logically complex function, NextDate
3. An example that typifies Management Information Systems (MIS) applications, known as the commission problem.

Integration and system testing uses three other examples:

1. A simplified version of an automated teller machine (ATM), known as the simple ATM (SATM) system;
2. The currency converter, an event-driven application typical of graphical user interface (GUI) applications
3. The windshield wiper control device from the Saturn™ automobile.

Finally, an object-oriented version of NextDate is provided, called OO-Calendar, which is used to illustrate aspects of testing object-oriented software.

The Triangle Problem

Problem Statement

Simple version: The triangle program accepts three integers, a, b, and c, as input. These are taken to be sides of a triangle. The output of the program is the type of triangle determined by the three sides: Equilateral, Isosceles, Scalene, or Not a Triangle.

Improved version: The triangle program accepts three integers, a, b, and c, as input. These are taken to be sides of a triangle. The integers a, b, and c must satisfy the following conditions:

- | | |
|--------------------------------------|------------------------------|
| c ₁ . $1 \leq a \leq 200$ | c ₄ . $a < b + c$ |
| c ₂ . $1 \leq b \leq 200$ | c ₅ . $b < a + c$ |
| c ₃ . $1 \leq c \leq 200$ | c ₆ . $c < a + b$ |

The output of the program is the type of triangle determined by the three sides: Equilateral, Isosceles, Scalene, or Not a Triangle. If an input value fails any of conditions c₁, c₂, or c₃, the program notes this with an output message, for example, "Value of b is not in the range of permitted values." If values of a, b, and c satisfy conditions c₁, c₂, and c₃, one of four mutually exclusive outputs is given:

If all three sides are equal, the program output is Equilateral.

If exactly one pair of sides is equal, the program output is Isosceles.

If no pair of sides is equal, the program output is Scalene.

If any of conditions c₄, c₅, and c₆ is not met, the program output is Not A Triangle

Discussion

This example contains clear but complex logic. It also typifies some of the incomplete definitions that impair communication among customers, developers, and testers. The first specification presumes the developers know some details about triangles, particularly the triangle inequality: the sum of any pair of sides must be strictly greater than the third side. The upper limit of 200 is both arbitrary and convenient

Traditional Implementation

Q 1.10) Explain the triangle problem statement along with flowchart for traditional implementation.

Q) Identify problem statement for a triangle with flowchart for traditional implementation.

The "traditional" implementation of this example has a Fortran-like style. The flowchart for this implementation appears in below figure

Program triangle1 'Fortran-like version

Dim a, b, c, match As Integer

Output ("Enter 3 integers which are sides of a triangle")

Input (a, b, c)

Output ("Side A is ", a)

Output ("Side B is ", b)

Output ("Side C is ", c)

match = 0

If a = b

 Then match = match + 1

EndIf

If a = c

 Then match = match + 2

EndIf

If b = c

 Then match = match + 3

EndIf

If match = 0

 Then If (a+b) <= c

 Then Output ("Not A Triangle")

 Else If (b + c) <= a

 Then Output ("Not A Triangle")

 Else If (a + c) <= b ' (10)

 Then Output ("Not A Triangle")

 Else Output ("Scalene")

```

                                EndIf
                        EndIf
                EndIf
Else  If match=1
    Then  If (a + c) <=b
        Then  Output ("Not A Triangle")
        Else  Output ("Isosceles")
    EndIf
Else  If match=2
    Then  If (a + c) <=b
        Then  Output ("Not A Triangle")
        Else  Output ("Isosceles")
    EndIf
Else  If match=3
    Then  If (b + c) <= a
        Then  Output ("Not A Triangle")
        Else  Output ("Isosceles")
    EndIf
    Else  Output ("Equilateral")
EndIf
EndIf
EndIf
End Triangle1
```

The variable "match" is used to record equality among pairs of the sides. A classical intricacy of the FORTRAN style is connected with the variable "match": notice that all three tests for the triangle inequality do not occur. If two sides are equal, ex: a and c, it is only necessary to compare $a + c$ with b. This observation clearly reduces the number of comparisons that must be made.

The efficiency of this version is obtained at the expense of clarity (and ease of testing). Notice that six ways are used to reach the Not A Triangle box, and three ways are used to reach the Isosceles box.

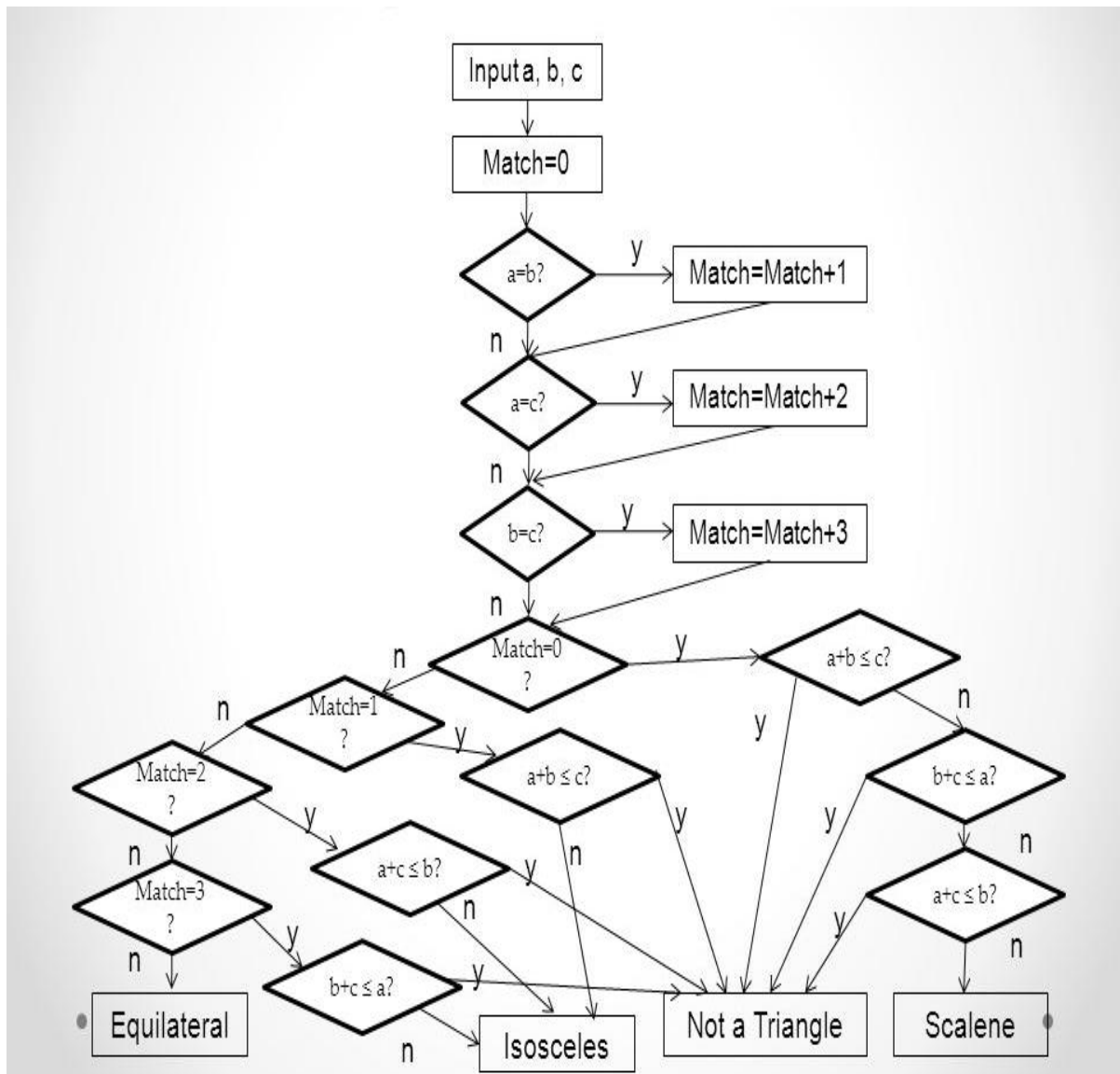


Figure 1.13: Flowchart of traditional implementation of triangle problem

Q 1.11) Discuss the Structured programming version of improved specification for triangle problem

Q) Explain structured implementation of triangle program with a neat dataflow diagram.

The following figure shows a dataflow diagram description of the triangle program. We could implement it as a main program with the three indicated procedures. The three procedures have been merged into one pseudocode program.

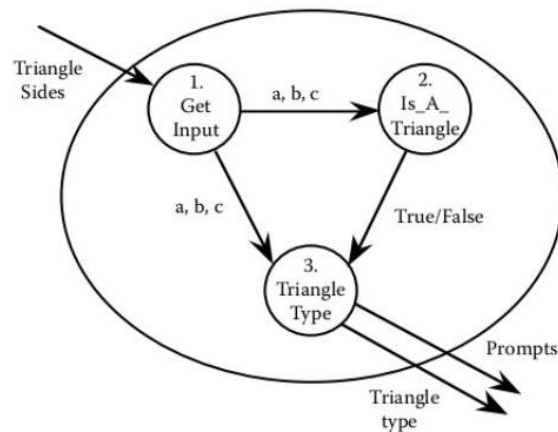


Figure 1.14 Dataflow diagram for a structured triangle program implementation

Program triangle2 'Structured programming version of simpler specification

```

Dim a, b, c As Integer
Dim IsATriangle As Boolean

```

Step 1: Get Input

```

Output ("Enter 3 integers which are sides of a triangle")
Input (a, b, c)
Output ("Side A is ", a)
Output ("Side B is ", b)
Output ("Side C is ", c)

```

Step 2: Is A Triangle?

```

If (a < b + c) AND (b < a + c) AND (c < a + b)
    Then IsATriangle = True
    Else IsATriangle = False
EndIf

```

Step 3: Determine Triangle Type

```

If IsATriangle
    Then If (a = b) AND (b = c)
        Then Output ("Equilateral")
        Else If (a ≠ b) AND (a ≠ c) AND (b ≠ c)
            Then Output ("Scalene")
            Else Output ("Isosceles")
        EndIf
    EndIf
Else Output ("Not a Triangle")
EndIf
End Triangle2

```

Program triangle3 'Structured programming version of improved specification'

Dim a, b, c As Integer

Dim IsATriangle As Boolean

Step 1: Get Input

Do

 Output ("Enter 3 integers which are sides of a triangle")

 Input (a, b, c)

 c1 = (1 <= a) AND (a <= 200)

 c2 = (1 <= b) AND (b <= 200)

 c3 = (1 <= c) AND (c <= 200)

 If NOT (c1)

 Then Output ("Value of a is not in the range of permitted values")

 EndIf

 If NOT (c2)

 Then Output ("Value of b is not in the range of permitted values")

 EndIf

 If NOT (c3)

 Then Output ("Value of c is not in the range of permitted values")

 EndIf

Until c1 AND c2 AND c3

Output ("Side A is ", a)

Output ("Side B is ", b)

Output ("Side C is ", c)

Step 2: Is A Triangle?

If (a < b + c) AND (b < a + c) AND (c < a + b)

 Then IsATriangle = True

 Else IsATriangle = False

EndIf

Step 3: Determine Triangle Type

If IsATriangle

 Then If (a = b) AND (b = c)

 Then Output ("Equilateral")

 Else If (a ≠ b) AND (a ≠ c) AND (b ≠ c)

 Then Output ("Scalene")

 Else Output ("Isosceles")

 EndIf

EndIf

Else Output ("Not a Triangle")

EndIf

NextDate Problem

Problem Statement

NextDate is a function of three variables: month, day, and year. It returns the date of the day after the input date. The month, day, and year variables have integer values subject to these conditions:

- c1. $1 < \text{month} < 12$
- c2. $1 < \text{day} < 31$
- c3. $1812 < \text{year} < 2012$

Discussion

Two sources of complexity exist in the NextDate function:

1. The complexity of the input domain
2. The rule that determines when a year is a leap year.

Implementation

Program NextDate1 'Simple version

Dim tomorrowDay, tomorrowMonth, tomorrowYear As Integer

Dim day, month, year As Integer

Output ("Enter today's date in the form MM DD YYYY")

Input (month, day, year)

Case month Of

Case 1: month Is 1,3,5,7,8, OR 10: '31-day months (except Dec.)

 If day < 31

 Then tomorrowDay = day + 1

 Else

 tomorrowDay = 1 tomorrow

 Month = month + 1

 EndIf

Case 2: month Is 4,6,9, OR 11 '30-day months

 If day < 30

 Then tomorrowDay = day + 1

 Else

```
tomorrowDay = 1 tomorrow
```

```
Month = month + 1
```

```
EndIf
```

Case 3: month Is 12: 'December

```
If day < 31
```

```
Then tomorrowDay = day + 1
```

```
Else
```

```
tomorrowDay = 1 tomorrow
```

```
Month = 1
```

```
If year = 2012
```

```
Then Output ("2012 is over")
```

```
Else tomorrow.year = year + 1
```

```
EndIf
```

```
EndIf
```

Case 4: month is 2: 'February

```
If day < 28
```

```
Then tomorrowDay = day + 1
```

```
Else
```

```
If day = 28
```

```
Then
```

```
If ((year is a leap year)
```

```
Then tomorrowDay = 29 'leap year
```

```
Else 'not a leap year
```

```
tomorrowDay = 1
```

```
tomorrowMonth = 3
```

```
EndIf
```

```
Else If day = 29
```

```
Then tomorrowDay = 1
```

```
tomorrowMonth = 3
```

```
Else Output ("Cannot have Feb.", day)
```

```
EndIf
```

```
EndIf
```

```
EndIf
```

EndCase

Output ("Tomorrow's date is", tomorrowMonth, tomorrowDay, tomorrowYear)

End NextDate

Q) Discuss the improved version of NextDate function

Dim tomorrowDay, tomorrowMonth, tomorrowYear As Integer

Dim day, month, year As Integer

Dim c1, c2, c3 As Boolean

Do

 Output ("Enter today's date in the form MM DD YYYY")

 Input (month, day, year)

 c1 = (1 <= day) AND (day <= 31)

 c2 = (1 <= month) AND (month <= 12)

 c3 = (1812 <= year) AND (year <= 2012)

 If NOT (c1)

 Then Output ("Value of day not in the range 1...31")

 EndIf

 If NOT (c2)

 Then Output ("Value of month not in the range 1...12")

 EndIf

 If NOT(c3)

 Then Output ("Value of year not in the range 1812...2012")

 EndIf

Until c1 AND c2 AND c3

Case month Of

Case 1: month Is 1,3,5,7,8, OR 10: '31-day months (except Dec.)

 If day < 31

 Then tomorrowDay = day + 1

 Else

 tomorrowDay = 1 tomorrow

 Month = month + 1

 EndIf

Case 2: month Is 4,6,9, OR 11 '30 day months

 If day < 30

```
    Then tomorrowDay = day + 1
  Else
    If day = 30
      Then tomorrowDay = 1
        tomorrowMonth = month + 1
      Else Output ("Invalid Input Date")
    EndIf
  EndIf
```

Case 3: month Is 12: 'December'

```
  If day < 31
    Then tomorrowDay = day + 1
  Else
    tomorrowDay = 1
    tomorrowMonth = 1
    If year = 2012
      Then Output ("Invalid Input Date")
      Else tomorrow.year = year + 1
    EndIf
  EndIf
```

Case 4: month is 2: 'February'

```
  If day < 28
    Then tomorrowDay = day + 1
  Else
    If day = 28
      Then
        If (year is a leap year)
          Then tomorrowDay = 29 'leap day
        Else 'not a leap year
          tomorrowDay = 1
          tomorrowMonth = 3
        EndIf
      Else
        If day = 29
          Then
            If (Year is a leap year)
              Then tomorrowDay = 1
                tomorrowMonth = 3
            Else
              If day > 29
                Then Output ("Invalid Input Date")
```

```
                                EndIf
                            EndIf
                        EndIf
                    EndIf
                EndIf
            EndCase
Output ("Tomorrow's date is", tomorrowMonth, tomorrowDay, tomorrowYear)

End NextDate2
```

The Commission Problem

Q) Discuss the commission problem with implementation

The third example is more typical of commercial computing. It contains a mix of computation and decision making, so it leads to interesting testing questions.

Problem Statement

- A rifle salesperson in the former Arizona Territory sold rifle locks, stocks, and barrels made by a gunsmith in Missouri.
Locks cost \$45, stocks cost \$30, and barrels cost \$25.
- The salesperson had to sell at least one complete rifle per month, and production limits were such that the most the salesperson could sell in a month was 70 locks, 80 stocks, and 90 barrels.
- After each town visit, the salesperson sent a telegram to the Missouri gunsmith with the number of locks, stocks, and barrels sold in that town. At the end of a month, the salesperson sent a very short telegram showing -1 lock sold.
- The gunsmith then knew the sales for the month were complete and computed the salesperson's commission as follows:
 - ✓ 10% on sales up to (and including) \$1000,
 - ✓ 15% on the next \$800, and
 - ✓ 20% on any sales in excess of \$1800.
- The commission program produced a monthly sales report that gave the total number of locks, stocks, and barrels sold, the salesperson's total dollar sales, and, finally, the commission.

Discussion

This problem separates into three distinct pieces: the input data portion, in which that could deal with input data validation, the sales calculation and the commission calculation portion.

Implementation

Program Commission (INPUT, OUTPUT)

Dim locks, stocks, barrels As Integer

Dim lockPrice, stockPrice, barrelPrice As Real

Dim totalLocks, totalStocks, totalBarrels As Integer

Dim lockSales, stockSales, barrelSales As Real

Dim sales, commission: REAL

lockPrice = 45.0

stockPrice = 30.0

barrelPrice = 25.0

totalLocks = 0

totalStocks = 0

totalBarrels = 0

Input(locks)

While NOT (locks = -1) 'Input device uses -1 to indicate end of data

 Input (stocks, barrels)

 totalLocks = totalLocks + locks

 totalStocks = totalStocks + stocks

 totalBarrels = totalBarrels + barrels

 Input(locks)

EndWhile

Output ("Locks sold: ", totalLocks)

Output ("Stocks sold: ", totalStocks)

Output ("Barrels sold: ", totalBarrels)

lockSales = lockPrice * totalLocks

stockSales = stockPrice * totalStocks

barrelSales = barrelPrice * totalBarrels

sales = lockSales + stockSales + barrelSales

Output ("Total sales: ", sales)

If (sales > 1800.0) Then

 commission = 0.10 * 1000.0

 commission = commission + 0.15 * 800.0

 commission = commission + 0.20 *(sales-1800.0)

Else If (sales > 1000.0) Then

 commission = 0.10 * 1000.0

 commission = commission + 0.15* (sales-1000.0)

 Else commission = 0.10 * sales

 EndIf

EndIf

Output ("Commission is \$", commission)

End Commission

The SATM System

Q) Explain the problem statement and discuss the SATM

The SATM system contains an interesting variety of functionality and interactions that typify the client side of client/server systems.

Problem Statement

The SATM system communicates with bank customers via the 15 screens shown in Figure 2.4. Using a terminal with features as shown in Figure 2.3, SATM customers can select any of three transaction types: deposits, withdrawals, and balance inquiries. These transactions can be done on two types of accounts: checking and savings.

When a bank customer arrives at an SATM station, screen 1 is displayed. The bank customer accesses the SATM system with a plastic card encoded with a personal account number (PAN), which is a key to an internal customer account file, containing, among other things, the customer's name and account information. If the customer's PAN matches the information in the customer account file, the system presents screen 2 to the customer. If the customer's PAN is not found, screen 4 is displayed, and the card is kept.

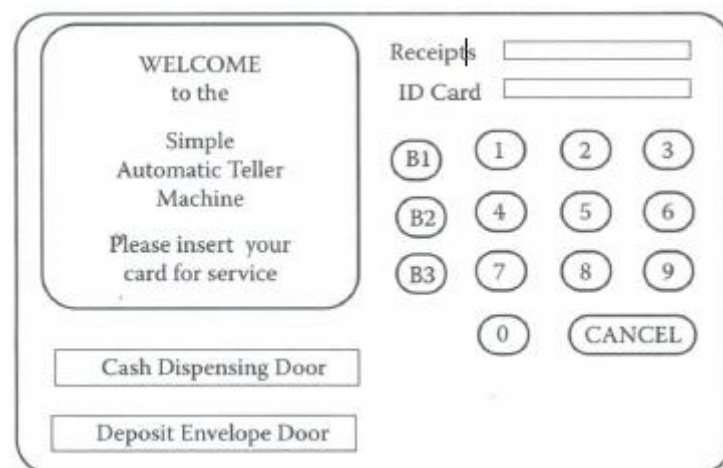


Figure 1.15 The SATM terminal

At screen 2, the customer is prompted to enter his or her personal identification number (PIN). If the PIN is correct (i.e., matches the information in the customer account file), the system displays screen 5; otherwise, screen 3 is displayed. The customer has three chances to get the PIN correct; after three failures, screen 4 is displayed, and the card is kept.

On entry to screen 5, the system adds two pieces of information to the customer's account file: the current date and an increment to the number of ATM sessions. The customer selects the desired transaction from the options shown on screen 5; then the system immediately displays

screen 6, where the customer chooses the account to which the selected transaction will be applied.

If balance is requested, the system checks the local ATM file for any un-posted transactions and reconciles these with the beginning balance for that day from the customer account file. Screen 14 is then displayed.

If a deposit is requested, the status of the deposit envelope slot is determined from a field in the terminal control file. If no problem is known, the system displays screen 7 to get the transaction amount. If a problem occurs with the deposit envelope slot, the system displays screen 12. Once the deposit amount has been entered, the system displays screen 13, accepts the deposit envelope, and processes the deposit. The deposit amount is entered as an unposted amount in the local ATM file, and the count of deposits per month is incremented. Both of these (and other information) are processed by the master ATM (centralized) system once a day. The system then displays screen 14.

If a withdrawal is requested, the system checks the status (jammed or free) of the withdrawal chute in the terminal control file. If jammed, screen 10 is displayed; otherwise, screen 7 is displayed so the customer can enter the withdrawal amount. Once the withdrawal amount is entered, the system checks the terminal status file to see if it has enough money to dispense. If it does not, screen 9 is displayed; otherwise, the withdrawal is processed. The system checks the customer balance (as described in the balance request transaction); if the funds are insufficient, screen 8 is displayed. If the account balance is sufficient, screen 11 is displayed and the money is dispensed.

The withdrawal amount is written to the unposted local ATM file, and the count of withdrawals per month is incremented. The balance is printed on the transaction receipt as it is for a balance request transaction. After the cash has been removed, the system displays screen 14.

When the "No" button is pressed in screen 10, 12, or 14, the system presents screen 15 and returns the customer's ATM card. Once the card is removed from the card slot, screen 1 is displayed. When the "Yes" button is pressed in screen 10, 12, or 14, the system presents screen 5 so the customer can select additional transactions.

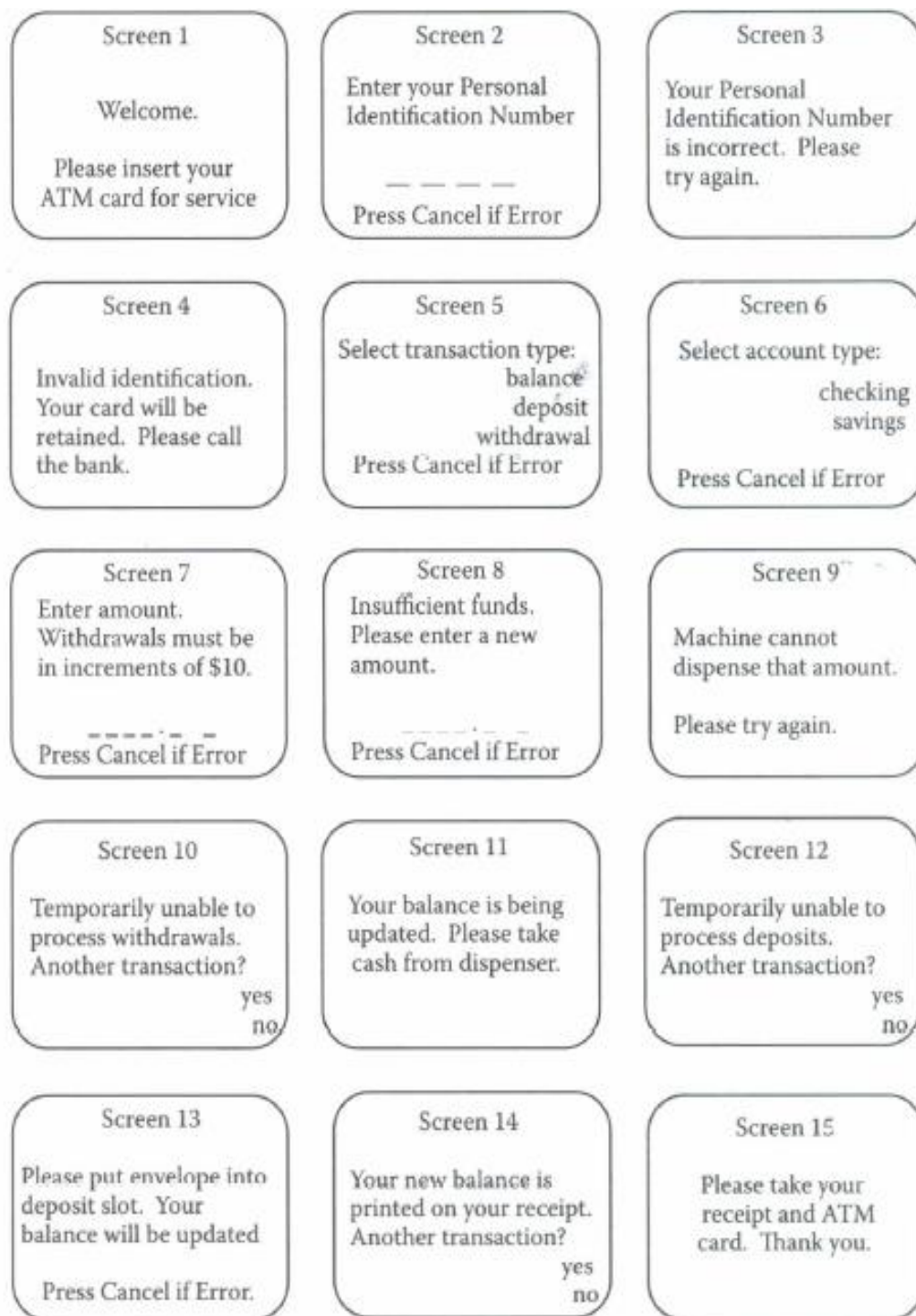


Figure 1.16: SATM Screens

Discussion

A surprising amount of information is "buried" in the system description just given. For instance, if you read it closely, you can infer that the terminal only contains \$10 bills (see screen 7). This textual definition is probably more precise than what is usually encountered in practice. The example is deliberately simple (hence the name).

The Currency Converter

Q 1.12.) With a neat diagram explain the currency conversion system

The currency conversion program is another event-driven program that emphasizes code associated with a graphical user interface (GUI). A sample GUI built with Visual Basic is shown in Figure 1.17.

- The application converts U.S. dollars to any of four currencies: Brazilian reals, Canadian dollars, European Union euros, and Japanese yen.
- Currency selection is governed by the radio buttons (Visual Basic option buttons), which are mutually exclusive. When a country is selected, the system responds by completing the label;
- By clicking on the Compute button results in the conversion of the U.S. dollar amount to the equivalent amount in the selected currency. Clicking on the Clear button resets the currency selection, the U.S. dollar amount, and the equivalent currency amount and the associated label. Clicking on the Quit button ends the application.



Figure 1.17: Currency Converter GUI

Saturn Windshield Wiper Controller

The windshield wiper on some Saturn automobiles is controlled by a lever with a dial. The lever has four positions — OFF, INT (for intermittent), LOW, and HIGH — and the dial has three positions, numbered simply 1, 2, and 3. The dial positions indicate three intermittent speeds, and the dial position is relevant only when the lever is at the INT position.

The decision table below shows the windshield wiper speeds (in wipes per minute) for the lever and dial positions.

c1.	Lever	OFF	INT	INT	INT	LOW	HIGH
c2.	Dial	n/a	1	2	3	n/a	n/a
a1.	Wiper	0	4	6	12	30	60

VTU Question Paper Questions

Q No.	Question	Marks	Year
1.	What is Software testing? Differentiate between functional testing and structural testing.	6	Dec-2019/Jan-2020
2.	Explain the triangle problem statement along with the flowchart for traditional implementation.	7 & 8	Dec-2019/Jan-2020 & June/July-2019
3.	Explain the several measures of software quality	3	Dec-2019/Jan-2020
4.	Define the terms: (i) error (ii) fault (iii) failure (iv) incident (v) test case	5	Dec-2019/Jan-2020
5.	With a neat diagram, explain the currency converter system	5	Dec-2019/Jan-2020
6.	With a neat diagram, summarize several strategies for test generation.	6	Dec-2019/Jan-2020
7.	What is software testing? Why it is so important in SDLC.	3	June/July-2019
8.	Explain the portrays of software testing life cycle	6	June/July-2019
9.	List six types of faults and explain each with example.	8	June/July-2019
10.	Describe the GUI application currency converter and embedded device Saturn wind shield wiper with diagram	8	June/July-2019
11.	Define the following: i) Error ii) Fault iii) Failure iv) Test v) Reliability vi) Usability vii) Correctness viii) Performance	8	June/July-2018
12.	Explain structured implementation of triangle problem with a neat dataflow diagram	8	June/July-2018
13.	Explain testing and debugging with a neat diagram	8	June/July-2018
14.	Explain functional testing and structural testing	8	June/July-2018

Prepared by

Usha G.R.
Assistant Professor
Dept. of ISE